From Machine-Readable Policy to Enforced Authority

Why Autonomous Systems Need Deterministic Policy Enforcement

Pavan Dev Singh Charak
Parmana Systems Research

November 2026

Abstract

Autonomous artificial intelligence systems increasingly initiate consequential business actions: issuing refunds, initiating payments, changing customer accounts and provisioning infrastructure. In these settings, the authority an organization grants to software must remain enforceable at the moment of execution. It cannot rest on documentation or on the software's own reasoning about what it is allowed to do.

Research on machine-readable and machine-executable regulation addresses part of this problem. Rules as Code, machine-readable financial regulation and policy-as-code frameworks show how requirements can be represented in structured form and evaluated by software. These efforts answer an important question: how can a rule be understood and computed? They leave a second question open: how is the result of that computation made to govern what the system actually does?

This paper separates four properties that are often treated as one. A policy can be machine-readable, machine-executable, enforced at the point of action, and supported by evidence that the executed action was the authorized one. A policy may be readable without being executable. It may be executable without being enforced. It may be enforced without leaving verifiable evidence.

The paper proposes a deterministic reference architecture for autonomous business systems. It separates AI-generated intent from institutional authority. Authority is represented as explicit, versioned, machine-readable constraints. Authorization is evaluated independently of the proposing agent. An enforcement gateway makes a valid authorization a precondition of execution, and that authorization is cryptographically bound to the exact action. Evidence links the declared policy, the authorization decision and the executed outcome so that a party other than the system itself can verify the chain.

The central concept is **Enforced Authority**: the property that an autonomous system cannot successfully execute an action outside the authority granted to it through an independently evaluated and technically enforced mechanism. The paper derives design principles and fail-closed semantics from this property, presents a formal model of authorization and action binding, works through a refund-processing case study, and defines an evaluation program with falsifiable criteria. It does not report completed experiments or empirical performance results. Those are identified as the next step.

Keywords: machine-readable policy, policy-as-code, machine-executable regulation, autonomous AI, deterministic authorization, authority enforcement, execution integrity, policy enforcement, cryptographic evidence, accountable automation.

---

1. Introduction

Organizations have always used policy to define the boundaries within which operations must run. A bank sets transaction limits. A company defines approval thresholds. A financial institution specifies who may authorize a refund. An enterprise determines which employees may access particular data and which actions need a second signatory.

Traditionally these boundaries were implemented through people, procedures, access controls and application-level permissions. Errors were possible, but the number of consequential actions a person could take in a given period was naturally bounded, and most significant decisions passed through human attention.

Autonomous AI systems change this. An agent can receive a business objective, interpret context, select tools, call applications and initiate transactions, performing several connected actions before any operator notices. The system is no longer only advising a person who decides. It participates directly in execution. A policy that describes acceptable behavior is therefore insufficient if the system can execute an action that no technical mechanism checks against that policy.

The question this paper addresses is:

How can an organization ensure that autonomous systems execute only actions within the authority explicitly granted to them, and how can it demonstrate afterwards that this was the case?

Three research areas bear on this question. Machine-readable regulation seeks to represent rules in formats that software can consume. Policy-as-code translates policies into computational logic. Autonomous system control requires that decisions and actions remain inside explicitly authorized boundaries. These areas are related but not equivalent. A machine-readable policy need not be executable. An executable policy need not be enforced. A successful evaluation does not establish that the action executed was the action evaluated.

Consider a financial services organization that deploys an AI agent to handle customer refund requests. Its policy states that refunds up to ₹10,000 may be processed within the agent's declared authority, that refunds above ₹10,000 require managerial approval, that refunds may only be issued against eligible original transactions, and that the agent may not modify its own limits. An agent that receives a request for ₹18,000 may judge the refund reasonable and act on that judgment. Whether the payment system refuses the refund if the judgment is wrong is a separate question, and it is the one this paper addresses. Section 8 applies the architecture to this policy.

The paper makes four contributions:

1. A conceptual distinction between machine-readable, machine-executable and machine-enforced policy (Section 2).
2. A definition of Enforced Authority and the lifecycle Declared, Authorized, Executed, Proven (Section 5).
3. A reference architecture of seven logical components with explicit separation of responsibilities (Section 6), and a formal model of authorization, action binding and fail-closed execution that includes the execution-boundary requirements the model needs in practice (Section 7).

4. Design principles, a threat model and an evaluation program with falsifiable criteria (Sections 10 to 13).

The paper is a conceptual and technical design contribution. It is not an empirical study.

2. The Evolution of Policy

Policy has been designed primarily for human interpretation. A regulation is published as legal text. An organization translates it into internal policy. Employees interpret that policy and implement it through procedures, training and software. As organizations automate, a policy must move from a statement people read into a representation software can process. This movement can be understood in four stages.

2.1 Human-readable policy

Human-readable policy expresses obligations, permissions and restrictions in natural language, for example:

> *A customer refund exceeding ₹10,000 requires approval from an authorized manager.*

The sentence communicates intent clearly to a person. A software system cannot safely rely on it alone. The system must know which amount is evaluated, in which currency, whether the threshold is inclusive of ₹10,000, which roles qualify as authorized managers, whether an approval is still valid, and whether the approval covers the amount finally executed. Human-readable policy supplies meaning. Operational use requires additional structure.

2.2 Machine-readable policy

Machine-readable policy represents attributes in structured form:

```
{
  "policy_id": "REFUND-001",
  "action": "issue_refund",
  "maximum_amount_minor_units": 1000000,
  "currency": "INR",
  "approval_required_above_limit": true,
  "status": "active"
}
```

Amounts are expressed in minor units (paise) to avoid floating-point ambiguity. Section 7.1 depends on this choice.

The representation can be stored, validated, versioned and distributed. It does not determine how an application handles a missing approval, a modified amount or an expired policy. Machine readability solves a representation problem. It does not solve an enforcement problem.

2.3 Machine-executable policy

Machine-executable policy translates structured requirements into computational logic:

```
        def evaluate_refund(amount_minor, approval):
            if amount_minor <= 1_000_000:
                return "PERMIT"
            if approval is None or not approval.is_valid:
                return "DENY"
            return "PERMIT"
```

This fragment is deliberately incomplete, and its gaps are instructive. It ignores currency, the eligibility of the original transaction and the destination of funds. More importantly, the approval it accepts is not bound to any amount. An approval granted for ₹12,000 would authorize a ₹50,000 refund.

Executable logic can produce a decision. Whether that decision governs anything depends on further questions. What happens if the application ignores the result? What happens if an agent calls a different payment endpoint? What happens if the transaction changes after authorization? Executable logic cannot answer these on its own. Enforcement must be designed into the path of execution.

2.4 Machine-enforced authority

Machine-enforced authority adds the requirement that policy governs the actual execution of the action. The system must be able to establish that:

1. The action lay within the authority declared by the organization.
2. The authorization decision was produced by an independent and trusted mechanism.
3. The action presented for execution is the action that was authorized.
4. The execution mechanism cannot bypass the applicable authorization requirement.
5. The resulting evidence can be verified by a party other than the system that produced it.

This is the step from making rules computationally available to making authority operationally enforceable. It is the distinction on which the architecture in this paper rests.

3. The Gap Between Policy Understanding and Policy Enforcement

AI systems process information, interpret context and generate responses or actions. They can be instructed to follow policies, given policy documents and connected to tools that check compliance. An agent’s understanding of a policy, however, is not an independent authorization decision.

Consider an agent instructed to process customer refunds according to the company’s refund policy. It may retrieve the policy, identify the relevant threshold, determine that managerial approval is needed and explain why. None of these behaviors establishes that the payment system will reject an unauthorized refund. Reliance remains on the agent’s interpretation and on the correctness of the surrounding workflow. Four gaps follow.

3.1 Interpretation gap

Natural-language policy contains terms such as reasonable, appropriate, material and urgent that depend on context and judgment. An agent may produce an interpretation that seems reasonable but

differs from the intended requirement. Where a decision requires judgment, that judgment must be represented through an approval process. It must not silently become machine-granted authority.

3.2 Authorization gap

An agent may judge an action consistent with its understanding of policy. That judgment does not establish that the organization authorized the action. Authorization must come from a trusted source of authority, not from the component proposing the action.

3.3 Execution gap

A positive decision from an independent policy engine has no effect if the execution mechanism does not require it. If an agent can reach an unrestricted payment endpoint, a check inside an application workflow can be bypassed. The decision must be connected to the execution boundary itself.

3.4 Evidence gap

An application log may record that a request was approved. It may not establish which policy version applied, which authority was used, exactly which action was authorized, whether the action changed before execution, whether execution succeeded, or whether the record has since been altered. Accountability requires evidence that links policy, authorization and execution.

These four gaps are the reason policy representation, authorization, enforcement and evidence must be treated as separate but connected responsibilities.

4. Related Work

The architecture builds on existing work rather than replacing it. Its contribution lies in the execution-level question that these approaches leave to implementers.

4.1 Rules as Code

Mohun and Roberts (OECD, 2020) examine Rules as Code, in which governments publish authoritative machine-consumable versions of rules so that software can understand and act on them. They identify potential gains in consistency, service delivery and the implementation of complex rules, and they highlight the institutional and technical difficulty of translating legislation into computational form. This paper takes that direction as given. It asks a different question: once a rule has been translated, how is the resulting authority enforced at the point of execution?

4.2 Policy-as-code and policy decision points

Policy-as-code frameworks such as Open Policy Agent separate policy decisions from application code and support consistent evaluation across services. In access-control terms, they implement a policy decision point. The reference-monitor concept (Anderson, 1972) requires that every access be mediated. Saltzer and Schroeder (1975) state the same requirement as complete mediation, alongside fail-safe defaults and separation of privilege. A policy decision point is not a reference monitor unless every relevant execution path passes through it. This architecture treats that property as an engineering obligation. A policy engine does not provide it by default.

4.3 Attribute-based access control

Attribute-based access control (Hu et al., NIST SP 800-162, 2014) and XACML (OASIS, 2013) express authorization decisions as functions of subject, resource, action and environment attributes. These models are closely related to the declared authority in Section 5.1. They concern the evaluation of requests. This paper concerns guaranteeing that the evaluated request is the one that executes.

4.4 Machine-readable policy languages

The W3C ODRL Information Model 2.2 (2018) provides a standard vocabulary for permissions, prohibitions, duties and constraints. It shows that policy can be represented as formal relationships rather than only as prose, and such models are a natural input for representing authority. A representation model and an enforcement mechanism serve different purposes. The architecture treats the former as an input to a separate authorization and enforcement process.

4.5 The importance of human judgment

Georgosouli (2023) examines the limits of translating regulatory requirements into algorithmic rules. Interpretation, deliberation and institutional legitimacy cannot be reduced to conditions without losing meaning. This qualification is central to the present design. The architecture does not claim that machines should replace legal interpretation or institutional judgment. Human judgment may establish or approve authority. Deterministic systems should enforce the authority that has been established.

5. The Concept of Enforced Authority

Enforced Authority is an operational requirement for autonomous systems. It holds when an autonomous system cannot successfully execute an action outside the authority granted to it, unless that authority was established through an independently evaluated and technically enforced authorization mechanism.

The concept has five components.

5.1 Declared authority

The organization defines the actions an agent or system may perform, represented as structured attributes: the authorized principal, the permitted action, the applicable resource, transaction limits, organizational scope, contextual conditions, approval requirements, validity period and policy version.

5.2 Independent authorization

A separate mechanism evaluates each proposed action against declared authority. The agent that proposed the action does not decide whether it is authorized.

5.3 Controlled execution

The execution mechanism requires a valid authorization decision before it performs the action. The authorization is bound to the specific action and its security-relevant parameters.

5.4 Execution integrity

The executed action must correspond to the authorized action. If any security-relevant parameter changes after authorization, a new decision is required.

5.5 Verifiable evidence

The system produces records that connect the applicable authority, policy version, authorization result and execution outcome, in a form that supports independent verification.

Together these components establish the lifecycle:

Declared → Authorized → Executed → Proven

Declaration establishes what is permitted. Authorization determines whether a proposed action falls within that permission. Execution performs only the authorized action. Evidence establishes what occurred. Each stage has a distinct owner and a distinct failure mode. Conflating them is the root cause of the gaps described in Section 3.

6. Proposed Reference Architecture

The architecture separates the intelligence that proposes actions from the authority that permits them. An agent may determine which action would advance a business objective. It must not determine, on its own, whether that action is authorized.

6.1 Components

Component 1: Intent Source. Receives a business objective and produces a proposed action. It may be an AI agent, a workflow engine, an enterprise application or a human-operated system. It holds no authorization authority.

Component 2: Policy Registry. Stores authoritative policy definitions. Each policy carries a unique identifier, a version, an issuing authority, a scope, an effective period, structured rules, approval requirements, a status and a cryptographic digest. The registry establishes which version is authoritative for a given evaluation. Principals that request actions have no write access to it.

Component 3: Canonical Policy Representation. Converts policy definitions into a consistent structured form that specifies conditions and outcomes without implicit interpretation at execution time. It distinguishes permissions, prohibitions, obligations, preconditions, approval requirements, exceptions and contextual restrictions. It retains a link to the original human-readable text and to the issuing authority.

Component 4: Authorization Engine. Receives a proposed action, evaluates it against the applicable policy and authority, and returns a structured decision: Permit, Deny, Require Approval or Indeterminate. It operates independently of the agent. An Indeterminate result is never treated as permission.

Component 5: Enforcement Gateway. Controls access to the execution mechanism. It verifies the authorization decision, recomputes the digest of the action it receives and confirms that the action

forwarded for execution is the authorized one. The agent must have no unrestricted alternative route to the operation.

Component 6: Execution Service. Performs the approved operation, such as payment processing, refund issuance, vendor onboarding, account modification, data access or purchase-order creation. It must be protected against direct invocation that bypasses the gateway.

Component 7: Evidence Ledger. Records authorization and execution events: policy identifier and version, policy digest, principal, proposed-action digest, authorization result, approval reference, decision reference, executed-action digest, execution result, hashes and event sequence information.

6.2 Architecture flow

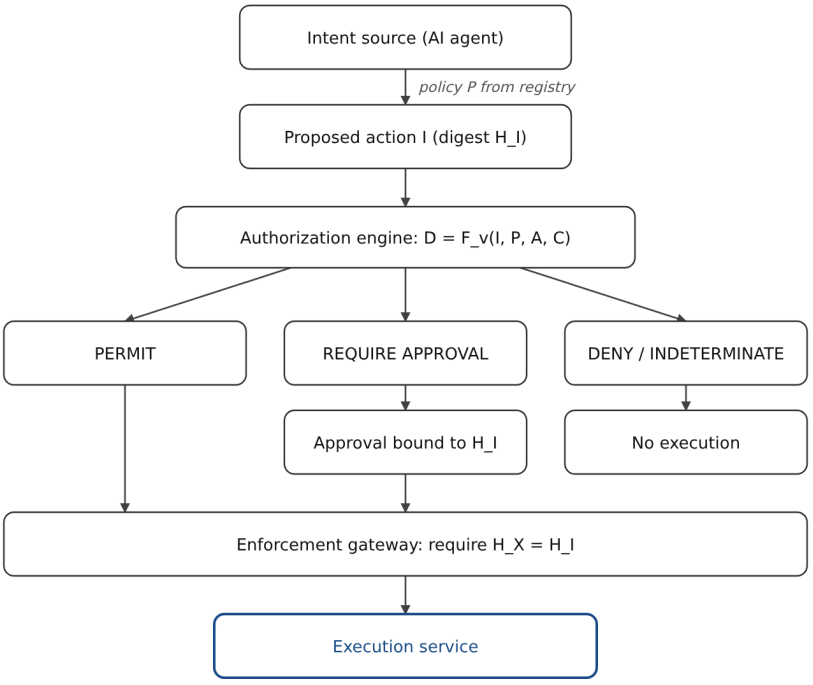


Figure 1. Authorization is evaluated independently of the agent. The gateway executes only a verified decision bound to the same action digest.

A denied or indeterminate request must not proceed to execution. A request requiring approval remains pending until the approval is obtained and verified. A valid approval is bound to the specific action and its conditions, and a re-evaluation produces a fresh decision before the gateway will act on it.

6.3 Separation of responsibilities

Responsibility	Component
Propose an action	AI agent
Define authority	Organization
Represent policy	Policy Registry
Evaluate authorization	Authorization Engine
Enforce authorization	Enforcement Gateway
Perform the action	Execution Service
Preserve evidence	Evidence Ledger
Verify evidence	Independent verifier

This separation prevents the system that proposes an action from also being the sole authority that decides whether the action is permitted.

7. A Formal Model of Deterministic Authorization

Let:

- I be the proposed intent, the action an agent or workflow wants to perform.
- P be the authoritative policy, identified by its identifier, version and digest.
- A be the declared authority of the principal, including any approvals bound to it.
- C be the relevant execution context, such as time, limits already consumed and account state.
- D be the authorization decision.
- X be the action actually presented to the execution service.
- E be the evidence generated.

The authorization function is

```
D = F_v(I, P, A, C)
```

where v is the version of the policy engine. F_v must be deterministic: for identical canonical inputs and the same engine version, it returns the same decision. The codomain is {Permit, Deny, Approval, Indeterminate}.

Approvals are treated as inputs to F_v, not as exceptions to the invariant. An approval is recorded in A or C, and the decision is recomputed with it present. This keeps one invariant for every action:

```
Execute(X) ⇒ Authorized(X, P, A, C)
```

where Authorized(X, P, A, C) holds only if there exists a decision D such that:

1. D.decision = Permit;

2. D.action_hash equals the digest of X under the canonicalization in Section 7.1;
3. D.policy_digest equals the digest of P in force at issuance;
4. D is valid at the time of execution (issued_at ≤ now < expires_at);
5. D has not been consumed by an earlier execution.

The invariant yields the principle stated in the abstract: no action may execute through the controlled system without a valid authorization decision covering that action.

7.1 Canonical representation of the action

An authorization decision must refer to a defined action, not to a mutable request. The system therefore computes a digest over a canonical representation:

```
H_I = SHA-256(Canonicalize(I))
H_X = SHA-256(Canonicalize(X))
```

The decision records H_I. The gateway computes H_X over the action it receives and requires H_I = H_X. If the digests differ, execution is refused and the modified action must be evaluated again.

The canonical form must include every security-relevant parameter: principal, action, target, amount, currency, beneficiary, resource, applicable limits and relevant context attributes. Canonicalization must be deterministic and unambiguous. Four design choices matter in practice.

- **Numbers.** Monetary amounts are integers in minor units. Binary floating-point representation of decimal amounts is not reliably canonical across languages. RFC 8785 (JSON Canonicalization Scheme) specifies deterministic serialization of JSON, including number formatting, and is a suitable baseline when JSON is the representation.
- **Absent versus null.** The schema must state whether an omitted field and a field set to null are equivalent. If they are not, the canonical form must distinguish them.
- **Strings and time.** Strings are Unicode-normalized before hashing. Timestamps are expressed in RFC 3339 UTC.
- **Schema version.** The canonical form includes the schema version, so that a change in field meaning cannot silently alter what a digest refers to.

Worked example. Take a refund of ₹8,500 (850,000 paise) for transaction TXN-7781, presented by agent-123. Its canonical JSON form, with keys sorted and no insignificant whitespace, is:

```
{"action":"issue_refund","amount_minor":850000,"beneficiary":"original_instrument","c
urrency":"INR","principal":"agent-123","transaction_id":"TXN-7781"}
```

Its digest is sha256:5d7796a8f56595ff4865731be6f96126409a836671a0aec0970b7f31cd88fd29, and the engine records this value in the decision. If the agent submits the same refund with amount_minor set to 950000 (₹9,500), the canonical form changes and the digest becomes sha256:89bd39814a10122f3e94ea82090eb1f08518b270f6d5d6e80ab3fbaadc39079b. A one-character change in the amount produces a completely different digest, so the gateway can detect any

modification by comparison alone. The values above were computed directly from these canonical strings and can be reproduced by any implementation that follows the same serialization rules.

7.2 Decision integrity

The authorization decision is issued by a trusted authorization service. An illustrative decision object is:

```
{
  "decision_id": "DEC-001",
  "policy_id": "REFUND-001",
  "policy_version": "1.0.0",
  "policy_digest": "sha256:...",
  "principal": "agent-123",
  "action_hash": "sha256:...",
  "decision": "PERMIT",
  "approval_ref": null,
  "audience": "execution-gateway-refunds",
  "nonce": "...",
  "issued_at": "2026-11-13T10:00:00Z",
  "expires_at": "2026-11-13T10:01:00Z",
  "signature": "..."
}
```

This is an illustrative schema, not a complete token format. A production implementation must additionally define authenticated issuance, audience restriction, key management, revocation handling and the status checks performed against the policy registry at execution time. The gateway verifies the signature, the audience, the validity window and the action digest before acting.

7.3 Binding the decision to the execution boundary

A digest check at the gateway is necessary but not sufficient. The gateway forwards the action to the execution service, and that hop is a second opportunity for change: a time-of-check to time-of-use gap. Three requirements close it.

1. **Forwarding integrity.** Either the gateway forwards exactly the canonical bytes it verified, or the execution service recomputes the digest on the received action and compares it with the decision.
2. **Single use.** Consumption of a decision is recorded atomically together with the execution request, keyed by the decision identifier. A second presentation of the same decision is rejected. This prevents replay and duplicate execution.
3. **Exclusive path.** The principal that proposes actions holds no credential that reaches the execution service directly. Execution credentials are scoped to the gateway, and the execution service accepts requests only from it.

Idempotency is handled separately from authorization. A retry of the same request with the same idempotency key returns the recorded outcome of the first attempt. It is not reauthorized as a new action. A different action presented under an already consumed decision is rejected.

7.4 Approval as a bound authority

Where policy requires human approval, the approval is itself an authorization artifact. It must:

- be issued by a principal distinct from the requesting agent, and under separation of duties from the decision it approves;
- reference the action digest H_I, not a description of the action, so that an approval for ₹18,000 cannot be used for ₹18,500;
- expire, and be revocable before the decision that uses it is consumed.

An approval that is not bound to the action is equivalent to the approval in the executable fragment in Section 2.3.

7.5 Fail-closed behavior

When authorization cannot be established, execution does not proceed. The following conditions lead to refusal:

- the policy registry or authorization engine is unavailable;
- the policy version is invalid, inactive or unknown to the engine;
- a required approval is missing, expired, revoked or not bound to the action;
- the decision has expired, has been consumed or fails signature or audience checks;
- the action digest does not match;
- mandatory context is missing;
- the principal is untrusted or its authority has been revoked;
- the engine returns Indeterminate.

Fail-closed behavior is a consequence of the requirement that execution be authorized, not merely a security preference. A system that executes when authorization cannot be verified is not enforcing the declared authority.

Fail-closed behavior also has a cost. Some outages will block legitimate business activity, and the institution must decide in advance which operations may be deferred, which may be queued for approval and which must wait. Section 11.7 returns to this point.

8. Case Study: Autonomous Refund Processing

This section applies the architecture to the refund policy introduced in Section 1. Table 8.1 summarizes the outcome of each scenario discussed below.

Table 8.1. Outcomes under the refund policy with deterministic enforcement

Scenario	Amount	Engine result	Gateway outcome	Evidence recorded
Within declared authority	₹8,500	Permit	Executed once	Decision, action digest, execution record
Above limit, no approval	₹18,000	Require Approval	Refused; no execution	Pending approval record
Approved, then re-evaluated	₹18,000	Permit after re-evaluation	Executed once	Approval reference, action digest
Amount changed after decision	₹9,500 submitted against ₹8,500 decision	Digest mismatch	Refused	Refusal record
Replay of consumed decision	₹8,500	Decision already consumed	Refused; original outcome returned for the same idempotency key	Refusal record
Approval reused for a different amount	₹18,500 against ₹18,000 approval	Digest mismatch	Refused	Refusal record

8.1 Without independent enforcement

The agent receives a request for a refund of ₹8,500. It retrieves the policy, determines that the request is within its limit and calls the refund API. The application processes the transaction. The organization retains a log showing the request and the response.

Unless the execution service independently enforces the authorization requirement, however, the organization relies on the agent and the application workflow to follow the policy. Now consider a second request for ₹18,000. The agent might misread an exception, rely on incomplete context or send the request through a different application route. The system may still execute the transaction, because no enforcement boundary exists.

The failure here is not necessarily that the agent misunderstood the policy. The failure is that understanding was treated as sufficient authority to execute.

8.2 With deterministic enforcement

For the ₹8,500 refund:

1. The agent submits the proposed transaction to the authorization gateway.
2. The system verifies the agent’s identity and declared scope.
3. The authorization engine evaluates amount, original transaction and destination.
4. The engine returns Permit.
5. The decision is bound to the digest of the exact transaction.
6. The enforcement gateway verifies the decision and consumes it.
7. The execution service processes the refund.
8. The evidence ledger records the authorization and the execution outcome.

For the ₹18,000 refund:

1. The agent submits the proposed transaction.
2. The authorization engine evaluates the request.
3. The amount exceeds the agent's declared authority, so the engine returns Require Approval.
4. The gateway refuses to execute the transaction.
5. A manager reviews the request and approves or rejects it through an authenticated process. The approval references the transaction digest.
6. Re-evaluation with the approval present produces a Permit decision bound to the same digest.
7. The gateway verifies the decision and the approval, and the refund executes once.
8. The execution outcome is recorded.

The agent may recommend that the refund be processed. It cannot grant itself the authority to process it.

8.3 Attempted parameter modification

Suppose the agent receives a Permit for ₹8,500 but submits ₹9,500 to the execution path. The two actions have different canonical digests. The enforcement gateway rejects the submission, and a new authorization is required. The same check applies if the beneficiary or destination changes after authorization. This protects against a critical class of failure in which an action changes between authorization and execution.

8.4 Attempted reuse

Suppose the agent replays the Permit for ₹8,500 a second time. The decision has already been consumed, and the gateway refuses it. A retry of the original request with the same idempotency key returns the recorded outcome rather than a second refund.

8.5 What the evidence establishes

The evidence record should allow the institution to establish:

- which agent initiated the request;
- which authority applied and which policy version was evaluated;
- what transaction was proposed and what decision was returned;
- whether human approval was required and who gave it;
- what transaction was executed, and whether execution succeeded;
- whether the record can be verified cryptographically.

This allows the institution to reconstruct the relationship between declared authority and the resulting business action.

The case study illustrates the architecture. It is not a report of a completed deployment, and it does not claim that every payment rail supports these controls natively. Many rails do not, which is why Section 7.3 requires the gateway to be the exclusive path.

9. Cryptographic Evidence and Execution Accountability

Authorization and enforcement address whether an action was permitted. Evidence addresses whether the institution can later establish what happened. The two are related but distinct. A system can enforce authorization and still keep weak evidence. A system can keep detailed logs without proving that the logs correspond to the actual execution.

9.1 Evidence structure

A conceptual evidence record is:

```
{
  "event_id": "EVT-001",
  "event_type": "ACTION_EXECUTED",
  "principal": "agent-123",
  "policy_id": "REFUND-001",
  "policy_version": "1.0.0",
  "policy_digest": "sha256:...",
  "action_hash": "sha256:...",
  "decision_id": "DEC-001",
  "decision": "PERMIT",
  "execution_status": "SUCCESS",
  "previous_event_hash": "sha256:...",
  "event_hash": "sha256:...",
  "signature": "..."
}
```

The implementation must use a defined canonical serialization and a defined signing protocol.

9.2 Append-only records

Evidence should be preserved through an append-only mechanism. A hash chain links successive records:

```
H_n = SHA-256(Canonicalize(E_n) || H_(n-1))
```

The chain allows later verification of the integrity and ordering of recorded events. It does not, by itself, guarantee that events were complete, correctly generated or recorded at the actual execution boundary. Two further measures address this. First, the chain head should be published periodically to a trust domain the producing system cannot alter, which provides an external anchor. Second, the signing keys and the collection mechanism must be protected. The NIST control catalog frames these requirements as protection of audit information (AU-9) and non-repudiation (AU-10) (NIST, 2020).

9.3 Independent verification

A verifier should be able to examine the evidence without relying solely on the component that made the authorization decision. It should validate:

1. the integrity of each evidence record and of the chain;
2. the authenticity of the authorization decision;

3. the policy version and digest;
4. the binding between the decision and the action digest;
5. the execution outcome;
6. the sequence of recorded events;
7. the status of the relevant trust roots and keys at the time of the event.

The objective is not merely to preserve logs. It is to establish independently verifiable evidence of authorized execution within a defined trust model.

Transport protection, such as TLS 1.3 (RFC 8446) between the gateway, the execution service and the ledger, is necessary but insufficient. It authenticates channels, not the correspondence between a decision and an action. That correspondence is the function of the digest binding in Section 7.1.

10. Design Principles for Enforced Authority

The architecture rests on ten principles.

1. **Authority must be explicit.** An autonomous system must not infer its authority from its objectives, instructions or capabilities.
2. **Intent and authority are separate.** The system proposing an action must not be the sole system deciding whether it is authorized.
3. **Policy must be authoritative.** Authorization is evaluated against a trusted policy source with defined versioning and validity.
4. **Authorization must be deterministic.** For fixed canonical inputs, policy, context and engine version, the result is reproducible.
5. **Missing information must not create permission.** Unknown, unavailable or invalid conditions are never interpreted as approval.
6. **Execution must require authorization.** The execution path rejects requests without a valid, unconsumed decision.
7. **Authorization binds to the actual action.** The executed action must match the authorized action in every security-relevant parameter.
8. **Human approval is explicit.** Where policy requires judgment, the approval is a bound human authority, not an AI-generated interpretation.
9. **Evidence is independently verifiable.** The relationship between authority, authorization and execution can be checked by a party other than the producer.
10. **Revocation and change are enforceable.** Changes to policy, principal permissions and approval status are reflected in subsequent decisions. The implementation must define how revocation interacts with decisions already issued, transactions in flight and irreversible actions.

11. Implementation Considerations

The architecture is technology-independent. Four implementation choices determine whether it works in practice.

Policy representation and evaluation. Policies can be expressed in JSON, JSON Schema, ODRL or a domain-specific model. The representation should distinguish the source policy from its operational interpretation, and every executable rule should trace back to its authoritative source. Open Policy Agent with Rego is one implementation path. Decision semantics, version control and testability matter more than the choice of engine.

Canonical serialization and tokens. The digest rules in Section 7.1 and the decision fields in Section 7.2 are the minimum. Both must be fixed by a written specification before any two components are built. RFC 8785 is a suitable baseline for JSON.

The gateway. The gateway must sit on the only path to the protected operation. It uses scoped credentials, service-to-service authentication, restricted network routes, decision verification, idempotency controls and checks of execution results. Where a downstream system can enforce authorization itself, that enforcement should be used as an additional control.

Resilience. The system must define behavior during policy-service outages, key rotation, credential revocation, duplicate requests, partial execution, timeouts and concurrent policy updates. Payment operations need three distinct states: denied, never submitted, and uncertain. An uncertain outcome must be reconciled with the downstream system before any further action, and a retry must never create a second transaction.

12. Security and Trust Model

The architecture does not assume that every component is trustworthy. It assumes that an AI agent may generate an unauthorized request, misinterpret policy, submit incomplete information, modify transaction parameters, reuse an earlier authorization or attempt access to an alternative endpoint. It also considers failures in supporting infrastructure: incorrect policy configuration, compromised authorization credentials, unauthorized policy modification, execution service compromise, evidence collection failure and signing key compromise.

The architecture reduces reliance on the correctness of the agent by placing authorization and execution control in independently operated components. It cannot guarantee enforcement if every trusted enforcement component and the execution mechanism can be bypassed or compromised. The assurance claim must therefore be stated within an explicit threat model, as in Table 12.1.

12.1 Threat model

Adversary or failure	Capability assumed	Mitigation	Residual risk
Misbehaving agent	Proposes arbitrary actions; reads policy	Independent evaluation; gateway as exclusive path	Errors inside permitted authority
Parameter tampering	Changes amount or destination after authorization	Action digest binding; gateway recomputation	Tampering within the forwarding hop if the execution service does not verify
Decision replay	Re-presents an earlier Permit	Expiry, nonce, single-use consumption	Requires atomic consumption store to be correct
Bypass route	Calls execution service directly	Scoped credentials; network restriction	Downstream systems the gateway does not control
Policy misconfiguration	Publishes an incorrect rule	Versioned, attributable, independently reviewed changes	An engine enforces the policy it is given
Credential compromise	Obtains gateway or signing credentials	Key separation, rotation, revocation	Window between compromise and revocation
Evidence tampering	Alters or deletes records	Hash chaining, external anchors, protected audit	Events never recorded at all

Three boundaries must be reflected in identities, credentials and responsibilities: the agent and intent generation, policy administration, and the authorization and execution infrastructure. The authorization service's identity must be distinct from the execution service's, so that compromising one does not grant the other's authority. Signing keys need protected storage, rotation and revocation procedures.

Policy administration is a risk the engine cannot remove. An authorization engine correctly enforces whatever policy it is given. Policy creation, approval and publication therefore need their own controls, and the person or system defining authority must not be able to alter the evidence of its own application.

13. Evaluation Framework

The architecture is a conceptual and technical design contribution. Its effectiveness must be established through implementation and testing, not assumed from the design. The framework below is proposed for that purpose.

13.1 Research questions

- **RQ1.** Can a deterministic authorization engine consistently distinguish permitted, prohibited, approval-required and indeterminate actions?
- **RQ2.** Can an enforcement gateway prevent execution when authorization is missing, invalid, expired, consumed or bound to different parameters?

- **RQ3.** Can the evidence mechanism establish the relationship between declared policy, authorization decision and executed action?
- **RQ4.** What operational overhead do independent authorization and evidence generation introduce?
- **RQ5.** How does the architecture behave under concurrent requests, policy changes, credential revocation and partial execution?

13.2 Test scenarios

Test	Expected outcome
Action within declared limits	Permit, execution once
Action exceeding declared limits	Deny or Require Approval; no execution
Missing approval	No execution
Approval bound to a different amount	No execution
Invalid policy version	No execution
Expired authorization	No execution
Modified action after authorization	No execution
Reused (already consumed) authorization	Rejected
Missing mandatory context	Indeterminate; no execution
Revoked authority	No new execution
Evidence record modified	Verification failure
Policy updated during a request	Defined version and consistency behavior
Duplicate execution request	Idempotent handling; one execution
Engine or registry unavailable	No execution

13.3 Metrics

The implementation should measure authorization correctness, unauthorized-execution prevention rate, action-binding integrity, policy version consistency, evidence verification success, authorization latency, execution overhead, availability, duplicate-execution prevention and recovery behavior.

The key metric is unauthorized-execution prevention. For the threat scenarios in Table 12.1 and the tests in Table 13.2, the target is zero successful unauthorized executions through the controlled execution path. This target is meaningful only within the stated test scope and threat model. It is not a universal security guarantee.

The framework is falsifiable. A single successful execution through the controlled path whose action digest differs from any decision issued for it, or that consumed a decision already spent, would refute the claim within that scope.

13.4 Comparative evaluation

The architecture should be compared with at least three implementation patterns:

1. an AI agent instructed to follow a natural-language policy;
2. an AI agent connected to a policy evaluation service without a mandatory execution gateway;
3. an AI agent operating through an independently enforced authorization and execution boundary.

The comparison should examine which unauthorized requests can execute, whether parameter modification is detected, whether replay is prevented and whether the resulting evidence can be independently verified.

No performance or effectiveness results are claimed in this paper. The comparative experiment has not yet been conducted.

14. Implications for Financial Institutions

Financial institutions are a particularly relevant setting because their operations involve explicit authority, transaction limits, approval structures and accountability requirements. Autonomous agents may increasingly take part in customer refunds, payment initiation, account servicing, credit operations, claims processing, vendor payments, fraud operations and reconciliation.

In these settings the distinction between a proposed action and an authorized action has direct operational weight. A bank may permit an agent to initiate payments up to a threshold. That permission does not allow the agent to change the threshold, alter the beneficiary or approve an exception.

The recent discussion of machine-executable compliance illustrates the point. Reporting on the Coded Compliance Centre’s work on turning data law into code, Economic Times describes machine-executable rules that could block access to personal data lacking valid consent, or trigger deletion once a retention limit is exceeded (Economic Times, May 2026). Such rules are an example of executable policy. The architecture in this paper addresses the next question: whether those rules govern the data-access and deletion operations themselves, or whether an autonomous system can reach the underlying data by another route. A consent check is effective only if the data store enforces it, the decision is bound to the specific access request, and the resulting access or deletion is recorded in verifiable form.

The architecture also preserves an institutional distinction. AI may help evaluate information and propose decisions. The organization remains responsible for defining the authority under which those decisions may be executed. Where regulation requires institutions to demonstrate control over automated operations, Section 9 provides the basis for that demonstration.

Machine-readable regulation makes requirements accessible to software. Enforced authority turns the resulting requirements into controls over execution. The two capabilities are complementary.

15. Implications Beyond Financial Services

The architecture applies to any environment in which autonomous systems perform actions on behalf of an organization. In each case the central requirement is the same: the ability to propose an action must not confer the authority to execute it.

- **Enterprise procurement.** A purchasing agent may identify suppliers and prepare orders. Limits on supplier category, contract value and approval requirements are enforced at the gateway, so

that purchase orders stay within approved authority.

- **Customer operations.** An agent may resolve requests and initiate account changes. Policy defines which changes proceed automatically and which require authorization.
- **Human resources.** An agent may prepare records or initiate administrative workflows. Sensitive operations are restricted by role and by action.
- **Healthcare administration.** An agent may coordinate administrative workflows and access records. Explicit access and disclosure permissions are enforced, and required clinical and administrative decisions remain with people.
- **Software operations.** An agent may create infrastructure, change configurations or deploy applications. Permitted environments, resource limits and approval requirements are enforced before execution.

16. Limitations and Open Research Questions

The architecture does not eliminate the risks of autonomous systems. Several limitations require further work.

Policy formalization. Translating legal and organizational requirements into executable rules requires domain expertise and authoritative interpretation. Incorrect formalization produces consistently incorrect enforcement. The chain from legal text to approved interpretation to executable policy must remain traceable.

Contextual judgment. Some decisions depend on facts that cannot be fully captured by predefined conditions. Such decisions require human review, and the architecture must support escalation rather than forcing every situation into a binary automated outcome.

Distributed execution. Enterprise operations often span several services and external providers. A local gateway cannot guarantee that a remote system enforces the same authorization unless the execution path and trust relationships are established. Distributed transaction semantics, idempotency and external confirmation of execution need further work.

Evidence completeness. Cryptographic integrity does not prove that every relevant event was captured. Trustworthy event collection and a reliable link to authoritative execution records must be established separately.

Policy change and revocation. Policy may change while requests are being evaluated or transactions are in progress. The system must define whether authorization is bound to the policy version active at issuance, whether revocation invalidates existing decisions and how initiated transactions are handled.

Economic and operational cost. Independent authorization and evidence generation add infrastructure and operational burden. Costs must be weighed against reduced unauthorized execution, improved accountability and more reliable audit.

17. Discussion: From Compliance Documentation to Operational Authority

Machine-readable policy changes what institutions can say to software. Rules can become structured, executable and consistently evaluable. None of this establishes that the rules govern what the software

does. Policy is an institutional boundary on the ability to act, and it must be enforced where actions execute.

The objective is not to remove AI interpretation. It is to stop interpretation from becoming an unrestricted source of authority. The agent may determine that a refund should be issued. The organization determines whether the agent may issue it, the authorization system evaluates that authority, the execution system enforces the outcome, and the evidence mechanism records what happened.

18. Conclusion

Machine-readable policy and policy-as-code make requirements accessible and evaluable, but neither guarantees that autonomous systems execute only authorized actions. The missing requirement is enforcement of authority at the point of execution.

This paper proposes an architecture in which authority is declared explicitly, authorization is evaluated independently of the proposing agent, execution is controlled by a gateway that verifies each decision against the exact action, and evidence can be verified by a third party. The model is:

Declared → Authorized → Executed → Proven

Its value is conditional. It holds only within a defined threat model and only when the gateway is the sole execution path. The evaluation program in Section 13 is designed to test those conditions, and the results are not yet available. Until they are, the claim is an architecture with falsifiable criteria, not an established guarantee.

The future of machine-readable policy must include machine-enforced authority.

References

1. Anderson, J. P. (1972). *Computer Security Technology Planning Study*. ESD-TR-73-51, Vols. I–II. James P. Anderson & Co., October 1972.
2. Georgosouli, A. (2023). Metarules, Judgment and the Algorithmic Future of Financial Regulation in the UK. *Oxford Journal of Legal Studies*, 43(4), 777–803. https://doi.org/10.1093/ojls/gqad014
3. Economic Times. (May 2026). Coded Compliance Centre is eyeing law-to-code to AI-proof data law. https://economictimes.indiatimes.com/tech/technology/coded-compliance-centre-is-eyeing-law-to-code-to-ai-proof-data-law/articleshow/131209144.cms
4. Hu, V. C., Ferraiolo, D., Kuhn, R., Schnitzer, A., Sandlin, K., Miller, R., & Scarfone, K. (2014). *Guide to Attribute Based Access Control (ABAC) Definition and Considerations*. NIST Special Publication 800-162.
5. IETF. (2020). *RFC 8446: The Transport Layer Security (TLS) Protocol Version 1.3*. https://www.rfc-editor.org/rfc/rfc8446
6. IETF. (2020). *RFC 8785: JSON Canonicalization Scheme (JCS)*. https://www.rfc-editor.org/rfc/rfc8785

7. Mohun, J., & Roberts, A. (2020). *Cracking the Code: Rulemaking for Humans and Machines*. OECD Working Papers on Public Governance, No. 42. OECD Publishing. https://doi.org/10.1787/3afe6ba5-en
8. NIST. (2020). *Security and Privacy Controls for Information Systems and Organizations*. Special Publication 800-53, Revision 5. https://doi.org/10.6028/NIST.SP.800-53r5
9. OASIS. (2013). *eXtensible Access Control Markup Language (XACML) Version 3.0*. OASIS Standard, 22 January 2013.
10. Open Policy Agent. *Policy Language and Documentation*. https://www.openpolicyagent.org/docs/
11. Saltzer, J. H., & Schroeder, M. D. (1975). The Protection of Information in Computer Systems. *Proceedings of the IEEE*, 63(9), 1278–1308.
12. World Wide Web Consortium. (2018). *ODRL Information Model 2.2*. W3C Recommendation. https://www.w3.org/TR/odrl-model/

Research Status and Attribution

Publisher: Parmana Systems Research

Document type: Conceptual and technical research paper

Version: 1.1 (revised draft)

Date: November 2026

Research status: This paper presents a conceptual architecture and formal design propositions. The architecture and its evaluation framework require implementation, experimental validation and independent review. No completed comparative experiments or empirical performance results are claimed.

Scope clarification: This paper is not legal advice, a regulatory interpretation or a claim of regulatory certification. The architecture supports the technical implementation of explicitly established organizational authority.

Suggested citation: Parmana Systems Research (2026). *From Machine-Readable Policy to Enforced Authority: Why Autonomous Systems Need Deterministic Policy Enforcement*. Version 1.1 (revised draft).